

Giáo Án

Phiên bản web: <https://chief-heliotrope-d20.notion.site/Gi-o-n-1c3b9e967cbc800db4cdee1f3abc2f19?pvs=4>

Quản Lý Trạng Thái Toàn Cục - Khi Nào Cần Redux Toolkit?

1. Mục tiêu

- Học viên nhận biết được sự khác biệt giữa trạng thái component cục bộ và trạng thái toàn cục.
- Học viên xác định được khi nào ứng dụng React cần đến giải pháp quản lý trạng thái toàn cục như Redux Toolkit.
- Học viên so sánh sơ lược Redux Toolkit với Context API và hiểu phạm vi sử dụng của từng công cụ.

2. Nội dung

2.1 Trạng thái Component Cục Bộ (Local State)

- Định nghĩa:** Trạng thái được quản lý bên trong một component bằng `useState`.
- Phạm vi:** Chỉ component đó và các component con trực tiếp (thông qua props).
- Phù hợp:** Trạng thái UI đơn giản, chỉ cần thiết cho một phần nhỏ của ứng dụng, không cần chia sẻ rộng rãi cho nhiều component.
- Ví dụ trong Todo App:**
 - Trạng thái input form trong component `TodoInput` (component nhập liệu todo).
 - Trạng thái toggle hiển thị menu tùy chọn (edit, delete) của một `TodoItem`.

2.2 Trạng Thái Toàn Cục (Global State)

- Định nghĩa:** Trạng thái được quản lý bên ngoài cấu trúc cây component, có thể được truy cập và thay đổi từ bất kỳ component nào trong ứng dụng.
- Phạm vi:** Toàn bộ ứng dụng.
- Phù hợp khi:**
 - Cần **chia sẻ trạng thái** giữa các nhiều component ở các nhánh khác nhau.
 - Ứng dụng **lớn và phức tạp** với nhiều tính năng và component.
 - Dữ liệu cần được **duy trì xuyên suốt** vòng đời ứng dụng, không bị mất khi component unmount.
 - Muốn **tổ chức code quản lý trạng thái** một cách rõ ràng, dễ bảo trì và mở rộng.
- Ví dụ trong Todo App:**
 - Danh sách Todos:** Nhiều component (ví dụ: `TodoList`, `TodoFilter`, `TodoStats`) cần truy cập và hiển thị danh sách todos.
 - Trạng thái bộ lọc:** Nếu bạn muốn thêm tính năng lọc todos (ví dụ: "completed", "incomplete", "all"), trạng thái bộ lọc nên là toàn cục để các component filter và list cùng truy cập.
 - Logic nghiệp vụ:** Logic thêm, sửa, xóa, toggle todo có thể phức tạp hơn và cần tổ chức tập trung.

2.3 Redux Toolkit vs. Context API - So Sánh Sơ Lược

Tính năng	Context API	Redux Toolkit
Độ phức tạp	Đơn giản	Ban đầu phức tạp, RTK giúp đơn giản hóa
Khả năng mở rộng	Tốt cho ứng dụng nhỏ đến vừa	Tốt hơn cho ứng dụng lớn và phức tạp
Hiệu năng re-render	Có thể kém khi context thay đổi thường xuyên	Tốt hơn với Selectors, tối ưu hóa re-render
Debug	Khó debug hơn	Dễ debug hơn với Redux DevTools
Trường hợp phù hợp	Trạng thái UI cục bộ, cấu hình (theme, ngôn ngữ)	Trạng thái toàn cục phức tạp, logic nghiệp vụ chính

- Context API:**

- Đơn giản, tích hợp sẵn trong React.
 - Phù hợp cho: Chia sẻ trạng thái **UI cục bộ hoặc các giá trị cấu hình** (theme, ngôn ngữ) ở phạm vi vừa phải.
 - **Hạn chế:** Khó quản lý logic phức tạp, re-render không tối ưu khi context thay đổi thường xuyên.
 - **Redux Toolkit:**
 - Mạnh mẽ, thư viện quản lý trạng thái chuyên dụng.
 - Phù hợp cho: Quản lý **trạng thái toàn cục phức tạp**, logic nghiệp vụ, dữ liệu ứng dụng chính.
 - **Ưu điểm:** Tổ chức code tốt, quản lý logic mạnh mẽ, công cụ debug tốt, tối ưu re-render với selectors.
-

Giới Thiệu Redux Toolkit và Các Khái Niệm Cốt Lõi

1. Mục tiêu

- Học viên hiểu được Redux Toolkit là gì và mục tiêu của nó.
- Học viên nắm vững các thành phần chính của Redux Toolkit: Store, Slice (Reducers & Actions)
- Học viên hiểu được luồng dữ liệu cơ bản trong Redux Toolkit.

2. Nội dung

2.1 Redux Toolkit - "Bộ Công Cụ" Hiện Đại Cho Redux

- Nhắc lại Redux là một thư viện quản lý trạng thái mạnh mẽ nhưng có thể **khó tiếp cận và cấu hình**.
- Redux Toolkit (RTK) ra đời để **giảm bớt sự phức tạp** khi sử dụng Redux, **giúp đơn giản hóa quá trình thiết lập** và cung cấp các **phương pháp tốt nhất** được khuyến nghị để quản lý trạng thái hiệu quả.
- RTK là một **tập hợp các hàm và công cụ** được thiết kế để giúp việc làm việc với Redux trở nên **dễ dàng và hiệu quả hơn** trong các dự án React.

2.2 Các Thành Phần Chính của Redux Toolkit

- **Store (Kho Lưu Trữ Trạng Thái):** Nơi **duy nhất** lưu trữ **toàn bộ trạng thái ứng dụng**. Đây là nguồn dữ liệu trung tâm mà các component có thể truy cập và cập nhật.
- **Slice (Lát Cắt Trạng Thái):**
 - Mỗi slice chịu trách nhiệm **quản lý trạng thái** và **logic nghiệp vụ** cho **một feature** hoặc **domain cụ thể** của ứng dụng. Tưởng tượng ứng dụng của bạn như một tòa nhà lớn, và mỗi slice là một phòng ban riêng biệt trong tòa nhà đó.
 - **Ví dụ:**
 - Trong một ứng dụng thương mại điện tử: bạn có thể có `productSlice` (quản lý danh sách sản phẩm, thông tin chi tiết sản phẩm), `cartSlice` (quản lý giỏ hàng), `userSlice` (quản lý thông tin người dùng, đăng nhập, đăng ký), `orderSlice` (quản lý đơn hàng).
 - Trong ứng dụng blog: bạn có thể có `blogSlice` (quản lý danh sách bài viết, bài viết chi tiết, categories), `commentSlice` (quản lý comments), `authorSlice` (quản lý thông tin tác giả).
 - Trong ứng dụng Todo: chúng ta có `todoSlice` (quản lý danh sách todos, trạng thái hoàn thành, v.v.).
 - 🚧 Hãy thoải mái phân chia slice sao cho hợp lý, đừng quá căng thẳng...nếu sai thì có thể sửa lại sau này.
 - Một Slice bao gồm:
 - **Reducer:** Xác định cách thức trạng thái của slice được cập nhật. Reducer là nơi **chứa logic để thay đổi trạng thái** dựa trên các actions.
 - **Actions:** Đại diện cho các sự kiện hoặc ý định muốn thay đổi trạng thái. Actions là cách để gửi tín hiệu đến **Reducer** để thực hiện việc cập nhật trạng thái.
 - **Initial State:** Trạng thái ban đầu của slice khi ứng dụng bắt đầu hoạt động.

2.3 Luồng Dữ Liệu trong Redux Toolkit

- Minh họa luồng dữ liệu bằng sơ đồ:

Component → Dispatch Action → Slice Reducer → Store → Component

- Giải thích luồng:
 1. **Component:** Khi một sự kiện xảy ra trong ứng dụng (ví dụ: người dùng nhấn vào nút **"Hiển thị thêm"** thì dữ liệu từ API trả về), component sẽ **dispatch (gửi đi) một action**.
 2. **Dispatch Action:** Action được gửi đến **Store**. Action này là một object JavaScript đơn giản mô tả **điều gì vừa xảy ra** hoặc **ý định thay đổi trạng thái**.
 3. **Slice Reducer:** Reducer **tương ứng** với action đó sẽ được kích hoạt. Reducer này:
 - Nhận vào **trạng thái hiện tại của slice** mà nó quản lý.
 - Dựa trên action và trạng thái hiện tại, reducer **tính toán và trả về một trạng thái mới** cho slice đó.
 4. **Store:** Store được cập nhật với **trạng thái mới** được trả về từ reducer.
 5. **Component:** Các component **"đang theo dõi"** (subscribe) đến phần trạng thái đã được cập nhật (thông qua `useSelector`) sẽ được thông báo về sự thay đổi trạng thái. Các component này sẽ **re-render** để phản ánh trạng thái mới trong giao diện người dùng.
- 📌 **Lưu ý cho giảng viên**
 - Giảng viên chuẩn bị trước sơ đồ và mô tả cho học viên hiểu sơ ý tưởng, không cần giải thích quá chi tiết. Mục đích chính là học viên **phải nắm rõ ý tưởng (ĐIỀU NÀY RẤT QUAN TRỌNG)**

Xây Dựng Store và Slice với Redux Toolkit

1. Mục tiêu

- Học viên nắm vững cách sử dụng `createSlice` để tạo slice reducer, actions và initial state cho `todoSlice`.
- Học viên biết cách sử dụng `configureStore` để thiết lập Redux Store cho ứng dụng Todo, sử dụng `todoSlice` đã tạo.
- Học viên hiểu rõ cách Redux Toolkit hỗ trợ viết reducer theo kiểu mutable nhưng vẫn đảm bảo immutability nhờ ImmerJS trong `todoSlice`.

2. Nội dung

📌 Lưu ý cho giảng viên:

- Giải thích cho học viên hiểu rõ khái niệm **mutable** và **immutability** trước khi đi tiếp.
- Tạo dự án React mới (vite) và cài đặt `@reduxjs/toolkit` và `react-redux`.

2.1 Tạo Slice với `createSlice` (`todoSlice` cho Todo App)

- Hàm `createSlice` giúp tạo slice reducer, action creators và action types tự động và ngắn gọn.
- Bắt đầu bằng việc tạo `todoSlice` trước, vì đây là phần logic quản lý trạng thái cụ thể:

```
// src/redux/slices/todoSlice.js
import { createSlice } from '@reduxjs/toolkit';

const todoSlice = createSlice({
  name: 'todos', // Tên slice, dùng để tạo action types và namespace cho slice state
  initialState: { // Trạng thái ban đầu của slice
    todos: [], // Mảng todos ban đầu rỗng
  },
  reducers: { // Object chứa các reducer functions, key là tên action, value là reducer function
    addTodo: (state, action) => { // Reducer function cho action "addTodo"
      // state: Trạng thái HIỆN TẠI của slice (KHÔNG phải toàn bộ store state)
      // action: Action object được dispatch, chứa payload (dữ liệu) nếu có
      const newTodo = action.payload;
      state.todos.push(newTodo); // MUTATE state trực tiếp! (ImmerJS lo immutability)
```

```

    },
    toggleTodo: (state, action) => { // Reducer function cho action "toggleTodo"
      const todold = action.payload;
      const todo = state.todos.find(todo => todo.id === todold);
      if (todo) {
        todo.completed = !todo.completed; // MUTATE state trực tiếp!
      }
    },
    // ... các reducer functions khác (ví dụ: deleteTodo, editTodo)
  },
});

// Action creators được tạo tự động
export const { addTodo, toggleTodo } = todoSlice.actions; // Ví dụ: { addTodo, toggleTodo } = { addTodo: function, toggleTodo: function }

// Export reducer của slice để kết hợp vào root reducer trong configureStore
export default todoSlice.reducer; // Export function reducer

```

- **Giải thích chi tiết `createSlice` :**

- `name` : Tên slice ('todos'), namespace cho action types (`todos/addTodo`).
- `initialState` : Trạng thái ban đầu (`{ todos: []}`).
- `reducers` : Object chứa reducer functions.
 - **Key** là tên action (`addTodo` , `toggleTodo`).
 - **Value** là reducer function (`(state, action) => { ... }`).
 - `state` : Trạng thái **HIỆN TẠI của SLICE (KHÔNG** phải toàn bộ store state)
 - `action` : Action object. `action.payload` chứa dữ liệu.
 - **MUTATE state TRỰC TIẾP** (ví dụ: `state.todos.push(newTodo)`). ImmerJS tự động xử lý chuyển sang **immutability**.

2.2 Thiết Lập Store với `configureStore`

- Hàm `configureStore` được dùng để tạo Redux Store một cách đơn giản và hiệu quả.
- Sau khi đã có `todoSlice.reducer` , chúng ta sẽ dùng nó để cấu hình Store.
- Cú pháp cơ bản:

```

// src/redux/store.js
import { configureStore } from '@reduxjs/toolkit';
import todoReducer from './slices/todoSlice'; // Import todoReducer từ todoSlice.js (File path: src/redux/slices/todoSlice.js)

const store = configureStore({
  reducer: { // Object chứa các slice reducers, key là tên slice, value là reducer
    todos: todoReducer, // Slice reducer cho feature "todos" - Sử dụng todoReducer đã tạo ở trên
    // ... các slice reducers khác nếu có (ví dụ: filterReducer, userReducer)
  },
});

export default store; // Export store để cung cấp cho ứng dụng React

```

- **Giải thích:**

- `reducer` : Thuộc tính bắt buộc, object chứa các slice reducers.
- **Key** của object là **tên slice** (ví dụ: `todos`), dùng để truy cập slice state trong component (`useSelector`).
- **Value** là **slice reducer** (`todoReducer`) đã được tạo bằng `createSlice` . Lưu ý: `todoReducer` được import từ `todoSlice.js` mà chúng ta đã tạo ở bước trước.

- `configureStore` tự động cấu hình store với middleware và Redux DevTools.
- 📌 **Lưu ý cho giảng viên**
 - Nhấn mạnh **mutate state trực tiếp trong reducer function** là hợp lệ nhờ ImmerJS trong `todoSlice` . Tại vì ImmerJS tự động xử lý chuyển sang **immutability**.
 - Giải thích action creators tự động và cách chúng được tạo ra từ `createSlice` .

Kết Nối Component với Redux Store và Selectors

1. Mục tiêu

- Học viên biết cách cung cấp Redux Store cho Todo App dùng `Provider` .
- Học viên nắm vững cách dùng `useSelector` Hook để truy cập trạng thái từ Redux Store trong component Todo App.
- Học viên hiểu Selector và `createSelector` để tối ưu hóa performance re-render trong Todo App.

2. Nội dung

2.1 Cung Cấp Store cho Ứng Dụng Todo với `Provider`

- Dùng `Provider` từ `react-redux` để component truy cập Redux Store.
- `Provider` làm cho Redux Store có sẵn cho **tất cả component con**.
- **Ví dụ:** Trong file `index.js` hoặc `App.js` :

```
// src/main.js
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App'; // File path: src/App.jsx
import store from './redux/store'; // Import store đã cấu hình (src/redux/store.js)
import { Provider } from 'react-redux';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <Provider store={store}> {/* Bọc App component bằng Provider và truyền store vào */}
      <App />
    </Provider>
  </React.StrictMode>
);
```

- **Giải thích:**
 - Import `Provider` và `store` .
 - Bọc `App` component bằng `<Provider store={store}>` .
 - `Provider` prop `store` nhận Redux Store từ `configureStore` .

2.2 Truy Cập Trạng Thái Store với `useSelector` Hook

- `useSelector` hook từ `react-redux` giúp component **"theo dõi"** trạng thái từ Redux Store và **re-render** khi thay đổi.
- Cú pháp cơ bản (Ví dụ trong component `TodoList`):

```
// src/components/TodoList/TodoList.jsx
import { useSelector } from 'react-redux';

function TodoList() {
  // useSelector nhận vào selector function
  const todos = useSelector((state) => state.todos.todos); // Selector function: (state) => state.todos.todos
```

```
return (  
  <ul>  
    {todos.map(todo => (  
      <li key={todo.id}>{todo.text}</li>  
    ))}  
  </ul>  
);  
}  
  
export default TodoList;
```

- **Giải thích `useSelector` :**
 - Import `useSelector` .
 - `useSelector` nhận **selector function** làm tham số.
 - **Selector function:**
 - `state` : Tham số đầu tiên là **toàn bộ Redux Store state**.
 - Trả về **phần trạng thái component muốn "theo dõi"** (`state.todos.todos` - mảng `todos` từ `todoSlice`).
 - `useSelector` **trả về giá trị** selector function trả về (mảng `todos`).
 - **Re-render:** Component re-render khi giá trị selector function trả về **thay đổi** (so sánh tham chiếu).
- 📌 **Lưu ý cho giảng viên**
 - Giải thích `Provider` cung cấp store cho Todo App.
 - Hướng dẫn `useSelector` truy cập state và re-render component Todo App.

Cập Nhật Trạng Thái với `dispatch` và Actions

1. Mục tiêu

- Học viên biết cách dispatch actions để cập nhật trạng thái Redux Store từ component Todo App.
- Học viên hiểu vai trò action creators và action types trong cập nhật trạng thái Todo App.
- Học viên nắm vững luồng dữ liệu từ component đến reducer qua dispatch và actions trong Todo App.
- Học viên hiểu cơ chế cập nhật tham chiếu trong Redux và **Tính chất lan truyền khi thay đổi tham chiếu**, cách kích hoạt re-render trong Todo App (📌 **QUAN TRỌNG**).

2. Nội dung

2.1 Dispatch Actions với `useDispatch` Hook

- `useDispatch` hook từ `react-redux` lấy hàm `dispatch` từ Redux Store.
- `dispatch` dùng để **gửi actions đến Redux Store** để cập nhật trạng thái Todo.
- Cú pháp cơ bản (Ví dụ trong component `TodoInput`):

```
// src/components/TodoInput/TodoInput.jsx  
import React, { useState } from 'react';  
import { useDispatch } from 'react-redux';  
  
import { addToDo } from '../redux/slices/todoSlice'; // Import action creator addToDo (File path: src/redux/slices/todoSlice.js)  
  
function TodoInput() {  
  const dispatch = useDispatch(); // Lấy hàm dispatch  
  const [todoText, setTodoText] = useState(''); // State cục bộ cho input text  
  
  const handleAddTodo = () => {
```



```

    if (todoText.trim() === '') {
      alert('Vui lòng nhập nội dung todo!');
      return;
    }
    const newTodo = { id: Date.now(), text: todoText, completed: false };
    dispatch(addTodo(newTodo));
    setTodoText('');
  };

  const handleInputChange = (e) => {
    setTodoText(e.target.value);
  };

  const handleInputChange = (e) => {
    setTodoText(e.target.value); // Cập nhật state cục bộ khi input thay đổi
  };

  return (
    <div>
      <input
        type="text"
        placeholder="Nhập todo mới..."
        value={todoText}
        onChange={handleInputChange}
      />
      <button onClick={handleAddTodo}>Add Todo</button>
    </div>
  );
}

// src/screens/Home/Home.jsx
import React from 'react';
import TodoInput from '../components/TodoInput/TodoInput'; // Import TodoInput component (File path:
src/components/TodoInput/TodoInput.jsx hoặc path tương ứng)
import TodoList from '../components/TodoList/TodoList'; // Import TodoList component (File path: src/c
omponents/TodoList/TodoList.jsx hoặc path tương ứng)

function Home() {
  return (
    <div>
      <h1>Ứng Dụng Todo với Redux Toolkit</h1>
      <TodoInput />
      <TodoList />
    </div>
  );
}

export default Home;

// src/App.jsx
import React from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Home from './screens/Home/Home';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
      </Routes>
    </BrowserRouter>
  );
}

```

```
    </BrowserRouter>
  );
}

export default App;
```

- **Giải thích `useDispatch` :**
 - Import `useDispatch` .
 - Gọi `useDispatch()` để lấy hàm `dispatch` .
 - `dispatch()` nhận **action object** làm tham số. Action object thường tạo bằng **action creators** (từ slice).
 - `dispatch(action)` gửi action đến reducer để xử lý và cập nhật trạng thái Todo.

2.2 Action Creators và Action Types (Trong Todo App)

- **Action Creators:** Hàm export từ slice (`todoSlice.actions.addToDo`).
 - Tạo action objects dễ dàng và type-safe. Ví dụ: `addToDo(payload)` , `toggleToDo(payload)` .
 - Trả về action object: `{ type: 'sliceName/actionName', payload: payload }` .
- **Action Types:** String constant xác định loại action (`'todos/addToDo'` , `'todos/toggleToDo'`).
 - Redux Toolkit tự động tạo action types dựa trên slice `name` và reducer function names.
 - Reducer dùng action types để xác định reducer function nào xử lý action.

2.3 Luồng Dữ Liệu Cập Nhật Trạng Thái (State)

- Quá trình cập nhật trạng thái trong Redux Toolkit diễn ra theo một luồng dữ liệu một chiều, dễ dự đoán:

Component → Dispatch Action → Slice Reducer → Store → Component Re-render (thông qua `useSelector`)

- Chi tiết từng bước:
 1. **Component:** Component trong ứng dụng React **chủ động kích hoạt quá trình cập nhật trạng thái** khi có một sự kiện xảy ra (ví dụ: người dùng click button, nhập liệu, hoặc dữ liệu từ API trả về). Component thực hiện việc này bằng cách **dispatch một action**.
 2. **Dispatch Action:** Component sử dụng hook `useDispatch` để lấy hàm `dispatch` và gọi hàm này, **gửi đi một action object** đến Redux Store. Action object này **mô tả điều gì vừa xảy ra hoặc ý định thay đổi trạng thái**.
 3. **Slice Reducer:** Redux Store nhận action và gửi nó đến **một slice reducer tương ứng**. Reducer này:
 - Nhận vào **trạng thái hiện tại (state) của slice** mà nó quản lý.
 - Dựa vào action và state hiện tại, reducer tính toán và trả về một trạng thái mới cho slice đó.
 4. **Store:** Redux Store **cập nhật trạng thái** của slice tương ứng bằng **trạng thái mới** mà reducer trả về. Lúc này, **toàn bộ store state (state toàn cục) đã được cập nhật**.
 5. **Component Re-render (thông qua `useSelector`):** Các component đã **"theo dõi"** (subscribe) đến phần trạng thái vừa được cập nhật (thông qua hook `useSelector`) sẽ được **thông báo về sự thay đổi trạng thái**. React Redux sẽ **so sánh tham chiếu (reference)** của giá trị trạng thái trước và sau khi cập nhật. **Nếu tham chiếu thay đổi, component sẽ re-render** để hiển thị dữ liệu mới.

2.4 Cơ Chế Cập Nhật Tham Chiếu và Tính Chất Lan Truyền Khi Thay Đổi Tham Chiếu

- **Tại sao cơ chế tham chiếu quan trọng?**
 - Trong React và Redux, việc **phát hiện thay đổi tham chiếu** là **then chốt để kích hoạt re-render** component một cách hiệu quả.
 - React Redux (thông qua `useSelector`) **so sánh tham chiếu** để xác định khi nào component cần re-render.
- **Redux Toolkit và ImmerJS đảm bảo cập nhật tham chiếu:**
 - Redux Toolkit (nhờ ImmerJS bên trong) **luôn đảm bảo việc cập nhật trạng thái là immutable**.

- Khi reducer trả về trạng thái mới, ImmerJS sẽ **tạo ra các object trạng thái mới (vùng nhớ mới)**, không sửa đổi object trạng thái cũ.
- Điều này dẫn đến việc **tham chiếu của object trạng thái luôn thay đổi** sau mỗi lần cập nhật.
- 📌 **Tính chất lan truyền khi thay đổi tham chiếu:**
 - Khi bạn cập nhật một phần dữ liệu con của state (ví dụ: thêm một object todo vào mảng `todos` trong `todoSlice`), Redux Toolkit (ImmerJS) sẽ tạo ra **tham chiếu mới cho toàn bộ dữ liệu chứa `todos` (dữ liệu cha, ông nội, ông cố,...)**
- **Ví dụ minh họa tính chất lan truyền:**
 - Nếu **state toàn cục** ban đầu là object lồng nhau như sau:

```
{ config: { theme: "light" }, user: { name: "Alice" } }
```
 - Khi bạn cập nhật **user.name**, ví dụ đổi từ "Alice" thành "Bob", Redux Toolkit sẽ tạo ra:
 - Một object **user mới** (tham chiếu mới) chứa `name` đã cập nhật thành "Bob".
 - Một object **state mới** (tham chiếu mới) chứa `config` (tham chiếu cũ - không thay đổi) và `user` (**tham chiếu mới**).
 - **Kết quả:** Tham chiếu của **state** và **user** thay đổi, nên kích hoạt re-render cho các component đang theo dõi **user** hoặc **state**. Còn Tham chiếu của **config** vẫn giữ nguyên, các component đang theo dõi **config** sẽ không re-render.
- 📌 **Lưu ý cho giảng viên**
 - **RẤT QUAN TRỌNG:** Giải thích chi tiết cơ chế cập nhật tham chiếu và **"Tính chất lan truyền khi thay đổi tham chiếu"**. Dùng ví dụ code và object trực tiếp minh họa

Giảng viên hướng dẫn hoàn thiện Toàn Bộ Ứng Dụng Todo

```
// src/main.js
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import store from './redux/store';
import { Provider } from 'react-redux';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </React.StrictMode>
);

// src/App.jsx
import React from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Home from './screens/Home/Home';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
      </Routes>
    </BrowserRouter>
  );
}
```

```

export default App;

// src/screens/Home/Home.jsx
import React from 'react';
import TodoInput from '../components/TodoInput/TodoInput';
import TodoList from '../components/TodoList/TodoList';

function Home() {
  return (
    <div>
      <h1>Ứng Dụng Todo với Redux Toolkit</h1>
      <TodoInput />
      <TodoList />
    </div>
  );
}

export default Home;

// src/components/TodoInput/TodoInput.jsx
import React, { useState } from 'react';
import { useDispatch } from 'react-redux';
import { addTodo } from '../redux/slices/todoSlice';

function TodoInput() {
  const dispatch = useDispatch();
  const [todoText, setTodoText] = useState('');

  const handleAddTodo = () => {
    if (todoText.trim() === '') {
      alert('Vui lòng nhập nội dung todo!');
      return;
    }
    const newTodo = { id: Date.now(), text: todoText, completed: false };
    dispatch(addTodo(newTodo));
    setTodoText('');
  };

  const handleInputChange = (e) => {
    setTodoText(e.target.value);
  };

  return (
    <div>
      <input
        type="text"
        placeholder="Nhập todo mới..."
        value={todoText}
        onChange={handleInputChange}
      />
      <button onClick={handleAddTodo}>Add Todo</button>
    </div>
  );
}

export default TodoInput;

// src/components/TodoList/TodoList.jsx
import React from 'react';

```

```

import { useSelector } from 'react-redux';
import TodoItem from '../TodoItem/TodoItem';

function TodoList() {
  const todos = useSelector((state) => state.todos.todos);

  return (
    <ul>
      {todos.map(todo => (
        <TodoItem key={todo.id} todo={todo} />
      ))}
    </ul>
  );
}

export default TodoList;

// src/components/TodoItem/TodoItem.jsx
import React from 'react';
import { useDispatch } from 'react-redux';
import { toggleTodo, deleteTodo } from '../redux/slices/todoSlice';

function TodoItem({ todo }) {
  const dispatch = useDispatch();

  const handleToggleTodo = () => {
    dispatch(toggleTodo(todo.id));
  };

  const handleDeleteTodo = () => {
    dispatch(deleteTodo(todo.id));
  };

  return (
    <li>
      <input
        type="checkbox"
        checked={todo.completed}
        onChange={handleToggleTodo}
      />
      <span style={{ textDecoration: todo.completed ? 'line-through' : 'none' }}>
        {todo.text}
      </span>
      <button onClick={handleDeleteTodo}>Delete</button>
    </li>
  );
}

export default TodoItem;

// src/redux/store.js
import { configureStore } from '@reduxjs/toolkit';
import todoReducer from './slices/todoSlice';

const store = configureStore({
  reducer: {
    todos: todoReducer,
  },
});

```

```

export default store;

// src/redux/slices/todoSlice.js
import { createSlice } from '@reduxjs/toolkit';

const todoSlice = createSlice({
  name: 'todos',
  initialState: {
    todos: [],
  },
  reducers: {
    addTodo: (state, action) => {
      state.todos.push(action.payload);
    },
    toggleTodo: (state, action) => {
      const todo = state.todos.find(todo => todo.id === action.payload);
      if (todo) {
        todo.completed = !todo.completed;
      }
    },
    deleteTodo: (state, action) => {
      state.todos = state.todos.filter(todo => todo.id !== action.payload);
    },
  },
});

export const { addTodo, toggleTodo, deleteTodo } = todoSlice.actions;
export default todoSlice.reducer;

```

-  **Lưu ý cho giảng viên:**
 - Có thể nói thêm về **Reselect (Tối Ưu Hóa Performance)** nếu cảm thấy học viên đã nắm vững kiến thức.
 - Khuyến khích học viên tự tìm hiểu thêm về Redux, tại vì **còn rất nhiều** kỹ thuật nâng cao hữu ích khác:
 - Trang chủ Redux Toolkit: <https://redux-toolkit.js.org/>
 - Tài liệu React Redux: <https://react-redux.js.org/>